

Rapport d'architecture - LOG430 (Format Arc42)

1. Introduction et objectifs

Ce projet vise à faire évoluer un système de caisse multi-magasins vers une architecture microservices, en suivant une approche orientée domaine et modulaire. L'objectif est de proposer la solution la plus simple répondant aux besoins du laboratoire 5, incluant observabilité, séparation des responsabilités, et évolutivité.

2. Contraintes

- Respect de la structure multi-magasin déjà existante
- Réutilisation de la base PostgreSQL
- Mise en place d'un monitoring Prometheus/Grafana
- Architecture conteneurisée avec Docker
- APIs REST exposées avec Express
- Redis utilisé pour le caching

3. Contexte

Le projet s'appuie sur les travaux des laboratoires précédents (Lab 1 à 3), où une application CLI a été construite avec persistance et logique métier. Le Labo 5 introduit la séparation en microservices :

- stock-service
- sales-service
- reports-service
- auth-service
- gateway

4. Stratégie de solution

- Refactor du monolithe en microservices indépendants
- Communication REST via HTTP (pas de messaging pour simplifier)
- API Gateway en façade unique (reverse proxy + authentification)
- Observabilité exposée sur chaque service via Prometheus
- Docker Compose utilisé pour l'orchestration locale

5. Vue des blocs fonctionnels

Structure des services :

```
/stock-service
├── src/
│   ├── api/
│   │   ├── controllers/
│   │   ├── middlewares/
│   │   └── routes/
```

```

|   |   └─ server.js
|   └── infrastructure/
|       └─ postgres/, redis/
|       └─ services/
|       └─ entities/

```

Chaque microservice suivra cette structure, adaptée à son domaine.

6. Vue dynamique (Runtime)

Exemple de requête :

- GET /api/v1/stock/:storeId passe par la Gateway → stock-service
- Le controller appelle un service, qui interagit avec le repository (Sequelize)
- Si cache manquant → lecture de PostgreSQL → réponse + mise en cache Redis

7. Vue de déploiement

Utilisation de docker-compose :

- Chaque microservice dans un conteneur
- Réseau pos-net partagé entre :
- PostgreSQL (pos_pg)
- Redis
- Prometheus + Grafana
- Gateway + services

Ports exposés pour développement local (ex: 3010 pour stock-service).

8. Concepts transversaux

- Caching Redis avec TTL par magasin (stock:store:{id})
- Logging via loggerMiddleware
- Monitoring via express-prom-bundle
- Authentification centralisée à terme dans la Gateway (JWT)
- ESM (type: module) activé pour tous les services

9. Décisions (ADR)

ADR-005 — Choix de KrakenD comme API Gateway

ADR-006 — Mise en place d'un Load Balancing

10. Scénarios de qualité

Critère	Scénario
Scalabilité	Chaque service peut être répliqué indépendamment

Critère	Scénario
Observabilité	Chaque service expose <code>/metrics</code> pour Prometheus
Maintenance	Structure claire par domaine, facile à isoler et debugger
Sécurité	Auth à centraliser dans la Gateway (à venir)

11. Risques et dettes techniques

Élément	État
Gateway non finalisée	✗ à faire
Swagger non intégré dans tous les services	✗ à faire
Authentification centralisée	✗ à faire
<code>sales-service</code>	✗ à créer
<code>reports-service</code>	✗ à créer
API création de comptes clients	✗ à créer
API panier d'achat	✗ à créer
API de validation de commande	✗ à créer
Tests automatisés	✗ à ajouter

12. Glossaire

Terme	Définition
ESM	ECMAScript Modules, syntaxe moderne JS (<code>import/export</code>)
ADR	Architectural Decision Record
TTL	Time To Live (durée de vie en cache)
Gateway	Façade unique qui redirige les appels vers les bons services
Prometheus	Outil de monitoring métrique
Grafana	Dashboard pour visualiser les métriques
Sequelize	ORM Node.js utilisé pour PostgreSQL